

# Course Outline: How Data Flows in a System

---

**Title:** How Data Flows in a System **Skill:** Skill 39: Building a data pipeline from the telemetry or user data **Learnpack:** + ¿Cómo fluyen los datos en un sistema? **File:** s39-w14d40-como-fluyen-los-datos-en-un-sistema **Prerequisite:** Payload Optimization (s37-w13d38-optimizacion-de-payload) **Target Audience:** Beginner developers building AI-powered FastAPI backends **Total Lessons:** 11 **Format:** Mixed (9% hands-on = 1 lesson)

---

## Course Description

Every app generates data — but raw data is useless to an AI model. Before a user click can improve a recommendation or a log entry can train a classifier, it must travel through a pipeline: collected from a source, ingested, transformed, and finally consumed by a model or delivered via an API. This course teaches students how that journey works, what happens at each stage, and why each stage exists — so they can design systems where data flows cleanly from source to output.

---

## Course Philosophy

This course emphasizes:

- Pipelines as design: data flow is an architectural decision, not an afterthought
  - Each stage exists for a reason: understanding *why* before implementing *how*
  - Best practices as defaults: reproducibility and isolation are not optional extras — they're what make pipelines reliable
- 

## Course Statistics Summary

| Section                            | Lessons   |
|------------------------------------|-----------|
| 00 - Welcome                       | 1         |
| 01 - Collecting and Preparing Data | 5         |
| 02 - Using Your Data               | 3         |
| 03 - Assessment                    | 1         |
| 04 - Outro                         | 1         |
| <b>Total</b>                       | <b>11</b> |

---

### 00.0 Welcome

#### Learning Objectives:

- Understand what a data pipeline is and why it exists

- Recognize the gap between raw app data and what an AI model can actually consume
- Preview the course structure and what each section covers

### **Content Outline:**

- The problem: your app generates data every second — but your AI model can't touch any of it without help
- What a data pipeline is: a series of stages that transform raw data into something a model (or API) can use
- Overview of the two main halves: collecting and preparing data vs. using that data
- What students will be able to explain by the end of this course

**Transitions:** Leads into Section 01, where we examine each stage of the input side of the pipeline — starting with where data actually comes from.

### **Key Principles:**

- Raw data is not model-ready data — the gap between them is what a pipeline bridges
- Every stage in a pipeline has a job: collect, clean, prepare, consume, deliver
- Understanding data flow is a prerequisite for building anything AI-powered

### **Examples:**

- A user clicks "add to cart" in an e-commerce app. That click is a raw event in a database — the AI model that will use it to improve recommendations doesn't see it yet. By the end of this course, you'll understand every step between that click and the model
- A team ships a feature that logs user sessions. Six months later, they want to train a churn prediction model. Where does that logged data go? How does it get there?

## 01.0 Collecting and Preparing Data

### **Learning Objectives:**

- Identify the three stages that make up the input side of a data pipeline: sources, ingestion, and transformation
- Understand what each stage does and why it's needed before data can be used
- Preview what lessons 01.1, 01.2, and 01.3 will each cover

### **Content Outline:**

- The input side of the pipeline: getting data from wherever it lives into a form a model can consume
- Stage 1 — Sources: the different places data lives in a system (users, integrations, storage)
- Stage 2 — Ingestion: moving data from its source into the pipeline
- Stage 3 — Transformation: cleaning, normalizing, and structuring data for use
- Why all three stages are necessary and why they're kept separate

**Transitions:** This overview frames the input side of the pipeline as a whole. Lesson 01.1 zooms into data sources — the starting point of any pipeline.

## Key Principles:

- The three input stages are not optional: skipping any of them means either losing data or feeding bad data to your models
- Each stage has a single job — when stages do multiple jobs, pipelines become fragile
- The output of each stage is the input of the next: sources feed ingestion, ingestion feeds transformation

## Examples:

- Analogy: think of the pipeline like a water treatment system. The source is the river (raw and mixed), ingestion is the intake pipe, and transformation is the filtration plant — only then is the water ready to use
  - A mobile app might have all three source types at once: user taps (behavior), a weather API (integration), and a user profile table (storage). Each needs different ingestion handling
- 

## 01.1 Data Sources

### Learning Objectives:

- Identify the three types of data sources in an application: user behavior, external integrations, and internal storage
- Explain what each source type produces and how it differs from the others
- Recognize which source type a given piece of data comes from

### Content Outline:

- Source type 1: User data — behavior (clicks, page views, time on screen) and interactions (form submissions, purchases, ratings)
- Source type 2: External integrations — third-party APIs, partner data feeds, bulk file imports (CSV, JSON)
- Source type 3: Internal storage — your own database as a data source (query historical records, export snapshots)
- Characteristics of each: volume, frequency, format, reliability
- Why knowing the source type matters: different sources require different ingestion strategies

**Transitions:** Now that we know where data comes from, lesson 01.2 covers ingestion — the act of moving it from those sources into the pipeline.

## Key Principles:

- Not all data sources are equal: user events are real-time and high-volume; database exports are batch and structured; integrations are unpredictable
- The source type determines how data enters the pipeline — ingestion is not one-size-fits-all
- Internal storage is often overlooked as a data source, but historical database records are frequently the richest training data a team has

## Examples:

- User behavior: `{"user_id": 42, "action": "click", "target": "add_to_cart", "ts": "2024-03-01T14:22:00"}`
  - External integration: a daily CSV from a logistics partner with shipment statuses
  - Internal storage: a SQL query that exports all orders from the past 90 days for churn analysis
- 

## 01.2 Ingestion

### Learning Objectives:

- Define ingestion and explain its role as the pipeline's entry point
- Distinguish between batch ingestion and streaming ingestion
- Explain why raw data is stored before transformation rather than transformed immediately

### Content Outline:

- What ingestion is: the process of pulling data from its source and delivering it to the pipeline for processing
- Batch ingestion: collecting data at scheduled intervals (e.g., every night, every hour) — good for high-volume, non-time-sensitive data
- Streaming ingestion: processing data as it arrives in real time — good for low-latency use cases
- The raw data store: why you save the original, unmodified data before touching it
- Ingestion is not transformation: the ingestion stage collects — it does not clean or structure

**Transitions:** With data in the pipeline's raw store, lesson 01.3 covers what happens next: transformation.

### Key Principles:

- Ingestion's only job is to move data — not to interpret or modify it
- Storing raw data before transformation is not wasteful: it means you can reprocess if your transformation logic changes
- Batch vs. streaming is a trade-off between simplicity and latency, not a quality difference

### Examples:

- Batch: a cron job runs every night at 2am, exports new user events from PostgreSQL, and writes them to a data lake as JSON files
  - Streaming: a message queue (like Kafka) receives click events from the app in real time, and a consumer reads and forwards them immediately
  - Raw store: the raw JSON files are never modified — if a bug in the transformer corrupts the processed data, the raw data is still there to reprocess
- 

## 01.3 Transformation

### Learning Objectives:

- Explain what data transformation is and why it's required before model training
- List common transformation operations: cleaning, normalizing, and structuring

- Apply the three core best practices for transformation: model isolation, reproducibility, and environment consistency

### Content Outline:

- What transformation is: converting raw, inconsistent data into a clean, structured format ready for model training
- Common operations: deduplication, handling missing values, normalizing types (strings to floats, mixed timestamps to ISO format), feature extraction
- Why models can't consume raw data directly: models expect consistent schemas, numeric types, and no nulls — raw data provides none of these guarantees
- Best practice 1 — Model isolation: models receive processed features, never raw data. The transformer is the firewall between source chaos and model expectations
- Best practice 2 — Reproducibility: running the same transform on the same raw data must always produce the same output. Non-reproducible transforms make bugs impossible to trace
- Best practice 3 — Environment consistency: the transform must run identically on a developer's laptop, a CI server, and a production machine. This is why dependencies are pinned and environments are containerized (introduced in Docker learnpack)

**Transitions:** Now that data is transformed, lesson 01.4 gives hands-on practice writing a transformation function.

### Key Principles:

- A transformation that only works on the machine it was written on is a liability, not an asset
- "Clean" means consistent types, no nulls (or explicit defaults), and a known schema — not just "no obvious errors"
- The output of transformation is the training dataset: what you put in determines what the model can learn

### Examples:

```
# Raw event – inconsistent types, missing fields, varied timestamp keys
raw = {"user_id": "42", "action": " Click ", "ts": "2024-03-01", "value":
None}

# After transformation – consistent, typed, model-ready
clean = {"user_id": 42, "action": "click", "timestamp": "2024-03-01",
"value": 0.0}
```

- Anti-pattern: transforming data at query time inside the model training script — if the script runs on a machine with a different locale, string parsing may produce different results, breaking reproducibility

**Challenge Prompt:** Write a `transform_events` function that takes a list of raw user event dictionaries and returns a cleaned, normalized list with consistent types and field names — then print the result.

**Solution Code:**

```
def transform_events(raw_events):
    transformed = []
    for event in raw_events:
        transformed.append({
            "user_id": int(event.get("user_id", 0)),
            "action": event.get("action", "unknown").lower().strip(),
            "timestamp": event.get("ts") or event.get("timestamp", ""),
            "value": float(event.get("value") or 0),
        })
    return transformed

events = [
    {"user_id": "1", "action": " Click ", "ts": "2024-01-01T10:00:00",
     "value": "3.5"},
    {"user_id": "2", "action": "VIEW", "timestamp": "2024-01-01T10:01:00",
     "value": None},
]

result = transform_events(events)
for e in result:
    print(e)
```

**Challenge Code:**

```
# TODO: Complete the transform_events function.
# It receives a list of raw event dicts and must return a cleaned, normalized
# list.
# Each output dict must have: user_id (int), action (lowercase stripped str),
# timestamp (from "ts" or "timestamp" key), value (float, default 0).

def transform_events(raw_events):
    transformed = []
    for event in raw_events:
        # your code here
        pass
    return transformed

events = [
    {"user_id": "1", "action": " Click ", "ts": "2024-01-01T10:00:00",
     "value": "3.5"},
    {"user_id": "2", "action": "VIEW", "timestamp": "2024-01-01T10:01:00",
     "value": None},
]
```

```
result = transform_events(events)
for e in result:
    print(e)
```

---

## 02.0 Using Your Data

### Learning Objectives:

- Identify the two stages on the output side of the pipeline: model consumption (training and inference) and delivery (storage and API)
- Understand how processed data moves from the transformation stage into model training and then to API consumers
- Preview what lessons 02.1 and 02.2 will each cover

### Content Outline:

- The output side of the pipeline: what happens after data is transformed
- Stage 4 — Training and Inference: models consume processed features to learn patterns (training) and generate predictions (inference)
- Stage 5 — Storage and API Delivery: predictions and processed data are stored, then served to consumers via an API
- How the two stages connect: a model trained on transformed data produces outputs that are themselves stored and delivered
- The pipeline as a loop: model outputs can become new data sources for the next pipeline run

**Transitions:** This overview frames the output side. Lesson 02.1 zooms into how models actually consume the data you've prepared.

### Key Principles:

- Transformation is the bridge between raw data and model consumption — without it, Section 02 doesn't exist
- Model outputs are data too: predictions stored in a database can feed future training runs
- The pipeline doesn't end at the model — it ends when value reaches the user (via the API)

### Examples:

- Transformed user events → training a recommendation model → model predicts next purchase → prediction stored in Redis → served via `/recommendations/{user_id}` endpoint
- The same processed features used for training can be queried by the inference endpoint at runtime — one transformation stage serves both

---

## 02.1 Training and Inference

### Learning Objectives:

- Explain the difference between model training and model inference

- Describe what data format models expect and why transformation is the prerequisite
- Identify why models must never consume raw data directly

### Content Outline:

- Training: feeding a processed dataset to a model so it learns statistical patterns — a batch process run periodically
- Inference: using a trained model to generate predictions on new input data — typically real-time or near-real-time
- What models expect: numeric tensors or feature vectors with a known, consistent schema — exactly what transformation produces
- Why raw data breaks models: inconsistent types, missing fields, and varied formats cause training errors or silently wrong predictions
- Training pipelines vs. inference pipelines: training is offline (scheduled), inference is online (on-demand)
- Feature stores: a shared store of processed features used by both training and inference to ensure consistency

**Transitions:** Once the model generates predictions, lesson 02.2 covers where those predictions go and how they reach the end user.

### Key Principles:

- A model trained on messy data learns messy patterns — garbage in, garbage out is a pipeline failure, not a model failure
- Training and inference must consume the same feature schema — if they diverge, predictions at inference time will be wrong
- Models are consumers, not processors: they receive clean data; they do not clean it themselves

### Examples:

- Training: a batch job runs every Sunday, loads the transformed user events from the last 30 days, and retrains the recommendation model
- Inference: a user opens the app, the `/recommendations` endpoint queries the trained model with the user's recent feature vector, and returns predictions in under 200ms
- Silent failure: a developer forgets to normalize `action` to lowercase before inference. The model was trained on lowercase actions. The prediction quality degrades, but no error is raised

## 02.2 Storage and API Delivery

### Learning Objectives:

- Identify what data gets stored at each stage of the pipeline and why
- Explain the role of an API in delivering model outputs to consumers
- Describe how storage choices at the output stage affect API performance

### Content Outline:



- What to store: raw data (for reprocessing), processed features (for training and inference), and model outputs/predictions (for delivery)
- Why store predictions instead of computing them on demand: latency, cost, and predictability
- The API layer: the endpoint that queries stored predictions or runs real-time inference and returns results to clients
- Storage options for predictions: relational DB (structured, queryable), key-value store like Redis (fast, ephemeral), or object storage (large outputs)
- The feedback loop: delivered predictions generate new user behavior → new data for the next pipeline run
- Monitoring: logging which predictions were served and whether users acted on them closes the loop for future training

**Transitions:** With the full pipeline understood from source to delivery, the assessment covers all five stages.

### Key Principles:

- Store predictions, not just model weights: computing predictions at request time is expensive and slow at scale
- The API is not the pipeline's end — user reactions to delivered predictions are the start of the next pipeline iteration
- Storage at the output stage is a performance decision, not just an architecture one: what you store and where determines how fast your API responds

### Examples:

- Stored predictions: after Sunday's training run, churn scores for all users are computed and stored in PostgreSQL. The `/churn-risk/{user_id}` endpoint reads from the table — no model inference needed per request
- Feedback loop: a user ignores a recommendation → that interaction is logged as a negative signal → ingested in the next batch → used to retrain the model
- Anti-pattern: computing model predictions inside the API handler on every request — fine for demos, catastrophic at scale when the model takes 2 seconds to load

## 03.0 Data Pipeline Assessment 🧠

### Learning Objectives:

- Demonstrate understanding of all five pipeline stages: sources, ingestion, transformation, training/inference, storage/delivery
- Apply best practices for transformation (reproducibility, model isolation, environment consistency)
- Identify correct pipeline design decisions in realistic scenarios

**Question format:** Scenario-based multiple choice

### Questions:

1. A user clicks "add to cart" in your app. The event is logged to PostgreSQL. A data scientist wants to use these events to train a recommendation model. What must happen first? a) Query the events table directly from the model training script b) Export the events as CSV and upload to the model manually c) ☒ Pass the events through ingestion and transformation to produce clean, model-ready features d) Train the model on raw SQL query results — modern frameworks handle raw data
2. Your ingestion job runs once per night. A stakeholder reports that the model's recommendations feel "stale" during peak hours. What is the most likely cause? a) The transformation step has a bug b) The API is returning cached responses c) ☒ Batch ingestion means the model trains on data from the previous day, not real-time events d) The storage layer is too slow
3. A developer's transformation script works perfectly on their laptop but produces different results on the CI server. Which best practice was violated? a) Model isolation b) Batch ingestion c) Feature extraction d) ☒ Environment consistency — the transform must produce identical results on any machine
4. Which is the correct definition of ingestion in a data pipeline? a) Cleaning and normalizing raw data before model training b) ☒ Moving data from its source into the pipeline for further processing c) Delivering model predictions to consumers via an API d) Training a model on a processed dataset
5. A team runs the same transformation script on the same raw dataset one month later and gets different output. Which best practice was violated? a) Model isolation b) Environment consistency c) ☒ Reproducibility — the same input must always produce the same output d) Feature extraction
6. Which statement about data sources is accurate? a) Only real-time user behavior data is suitable for AI model training b) Raw database exports can be fed directly into model training without transformation c) External integrations are too unreliable to use in production pipelines d) ☒ Data can come from user behavior, external integrations, and internal storage — each requiring different ingestion strategies
7. After your model generates churn predictions for all users, where should those predictions go? a) Back into the raw events table b) Directly to the front-end without any storage c) Into the transformation layer for further cleaning d) ☒ Into a database or cache, then served to consumers via an API endpoint

#### **Evaluation criteria:**

- 7/7: Excellent — you understand the full data pipeline from source to delivery
- 6/7: Strong — review the concept for the question you missed
- 5/7 or below: Return to lessons 01.1–02.2 before moving on

#### **Score interpretation:**

- 7/7 — You can trace data from a raw app event all the way to a model-powered API response.
- 6/7 — Solid foundation with one gap — identify which stage and review it.
- 5/7 or below — The pipeline stages are blurring together. Revisit the course before continuing.

---

## 04.0 Build Systems That Flow

### Content Outline:

- Course recap: what students accomplished — tracing the full journey from raw event to API-delivered prediction
- The five stages mastered: sources → ingestion → transformation → training/inference → storage/API
- Connection to bigger picture: data pipelines are the infrastructure that makes AI applications possible — without them, there are no AI features, only raw data sitting in a database
- Next steps: upcoming learnpacks cover storage formats and database selection (Skill 40), telemetry architecture (Skill 41), and frontend telemetry (Skill 42) — each builds on the pipeline foundation established here
- Encouragement: most developers never think about where their model's training data comes from. You now can — and that's a significant engineering advantage

**Note:** This is conclusion only — no theory, exercises, or assessment.